

Running Quantum Jobs on Q50 via LUMI

Event participant guide — accessing the Q50 quantum computer through the LUMI web interface

This guide walks you through running quantum programs on **Q50**, a superconducting quantum computer, from the **LUMI** supercomputer. For this event we have prepared a dedicated software module and a LUMI reservation so you can get started quickly.

NOTE

Two things have been set up for you:

1. A custom module, `fiqci-vtt-qiskit-JQH`, with Qiskit and the Q50 tooling pre-installed.
2. A reservation, `JQH2026`, that gives you priority access to compute during the event.

Before you begin

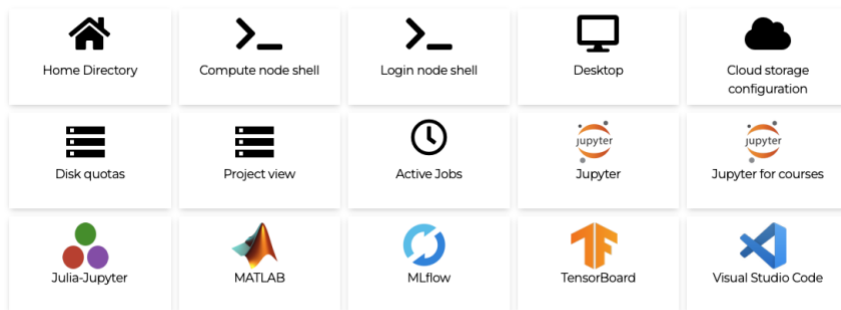
- A **Puhuri** account with access to the event project.
- A web browser. Everything runs in the browser, nothing to install on your own machine.
- These are the values used throughout this guide:

Setting	Value
Reservation	<code>JQH2026</code>
Project	<code>project_465003017</code>
Partition	<code>debug (max 30 min per session)</code>
Modules to load	<code>Local-quantum, fiqci-vtt-qiskit-JQH</code>

Step 1 · Log in and open Jupyter

1. Go to the [LUMI web interface](#) and sign in with your **Puhuri** account.
2. On the dashboard, find the **Pinned Apps** panel and click **Jupyter**.

Pinned Apps



The LUMI dashboard — select the “Jupyter” app.

Step 2 · Configure your session

On the Jupyter launch form, set the following fields:

- **Reservation** → choose **JQH2026**.
- **Project** → select the event project (**project_465003017**).
- **Time** → up to **00:30:00**. The reservation uses the **debug** partition, which is limited to 30 minutes per session.
- **Working directory** → set this to your scratch directory and create a subfolder for your work (for example **/scratch/project_465003017/yourname**). This is where your notebooks will be saved.
- **Script or path to script** → paste the two module-load lines below.

```
module load Local-quantum
module load fiqci-vtt-qiskit-JQH
```

Home / My Interactive Sessions / Jupyter

Saved Settings

You have no saved settings.

Apps

Editors

- Visual Studio Code

Graphical applications

Desktop

Servers

- MLflow
- Julia-Jupyter
- MATLAB
- TensorBoard
- Jupyter**

Tools

- Compute node shell

Course environments

- Jupyter for courses

Jupyter

Interactive Jupyter session

Documentation

Reservation

✓ No reservation
JQH2026 (active from 2026-06-05 13:00 UTC)

Project

project_465003077 (LUST Training / 2026-06-05-07 Junction Quantum)

The selected project will be available as **PROJECT** in other fields in this form.

Partition

debug

Resources

Number of CPU cores

1

SMT is enabled for the selected partition. 2 threads per core will be allocated.

Memory (GiB)

1

Time

00:30:00

d-hh:mm:ss, or hh:mm:ss

Settings

Working directory

/scratch/project_465003077

Basic Advanced

Custom Python type

Script

Module version

cray-python/3.11.7 (default)

Specify the version of the module you would like to use.

Script or path to script

module load Local-quantum
module load fiqi-vtt-qiskit-JQH

The launch form — set the reservation, project, time, working directory and module lines.

TIP

Want more than 30 minutes, or a larger node? Set **Reservation** to **No reservation** and pick a different partition. See [LUMI partitions](#) for the options. Note that jobs outside the reservation may queue for longer.

Step 3 · Launch and connect

1. Click the blue **Launch** button at the bottom of the form.

00:30:00
d-hh:mm:ss, or hh:mm:ss

Settings

Working directory

/scratch/project_465003017

Basic Advanced

Custom Python type

Script

Module version

cray-python/3.11.7 (default)

Specify the version of the module you would like to use.

Script or path to script

module load Local-quantum
module load fiqi-vtt-qiskit-JQH

The provided script will be run before starting Jupyter. If a path to a script is provided, it will be sourced by bash before starting Jupyter. Ensure that the script sets up the environment so that **python** is in **PATH** and Jupyter is installed.

☐ Save settings

Save settings and close

Launch

Reset to default settings

Scroll to the bottom of the form and click “Launch”.

- Your session is now queued. Depending on demand this takes a few seconds to a few minutes.
- When the card turns **Running**, click **Connect to Jupyter**. JupyterLab opens in a new browser tab.

Home / My Interactive Sessions

Saved Settings
You have no saved settings.

Apps
Editors
Visual Studio Code
Graphical applications
Desktop
Servers
MLflow
Julia-Jupyter
MATLAB
TensorBoard
Jupyter
Tools
Compute node shell
Course environments
Jupyter for courses

Jupyter (19020797)
1 node | 2 cores | Running

Host: >_ nls002278
Created at: 2026-06-03 09:42:03 UTC
Time Remaining: 29 minutes
Session ID: 6935047b-b6db-4b3a-9a4a-38bfcd4f52f
If you run into issues, please include the following log file in the support ticket: [output.log](#)
Project: project_465003017
Partition: debug
Cores: 2
Memory: 1024M
Connect to Jupyter

A running session — click “Connect to Jupyter” to open your notebook.

Step 4 · Run your first quantum program

In JupyterLab, create a new notebook and try the example below. It builds a **Bell pair** (a simple two-qubit entangled state) runs it on Q50, and prints the measurement results.

```
sample_job.ipynb
[1]: import os
    from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
    from qiskit_ismq import IQMProvider

[2]: Q50_CORTEX_URL = os.getenv('Q50_CORTEX_URL')

[3]: provider = IQMProvider(Q50_CORTEX_URL, quantum_computer='q50')
    backend = provider.get_backend()

[4]: shots = 1000

    qreg = QuantumRegister(2, 'QB')
    circuit = QuantumCircuit(qreg, name='Bell_pair')

    circuit.h(qreg[0])
    circuit.cx(qreg[0], qreg[1])
    circuit.measure_all()

[5]: transpiled_circuit = transpile(circuit, backend)
    transpiled_circuit.name = 'Bell_pair'

[ ]: job = backend.run(transpiled_circuit)
    result = job.result()

[ ]: counts = job.result().get_counts()
    print(counts)
```

A sample notebook that runs a Bell-pair circuit on Q50.

The same code as text, so you can copy it:

```
import os
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit_ismq import IQMProvider

# The module sets this environment variable for you
Q50_CORTEX_URL = os.getenv('Q50_CORTEX_URL')
provider = IQMProvider(Q50_CORTEX_URL, quantum_computer='q50')
backend = provider.get_backend()

shots = 1000
qreg = QuantumRegister(2, 'QB')
circuit = QuantumCircuit(qreg, name='Bell_pair')
circuit.h(qreg[0]) # superposition
circuit.cx(qreg[0], qreg[1]) # entangle the two qubits
circuit.measure_all()

transpiled = transpile(circuit, backend)
job = backend.run(transpiled, shots=shots)
counts = job.result().get_counts()
print(counts)
```

TIP

Save your work from the top-left menu: **File** → **Save Notebook** (one file) or **Save All** (everything). Files are written to the working directory you chose at launch.

Running jobs from the terminal (sbatch)

For longer or unattended runs you can submit a batch job. Open a terminal, either **Login node shell** on the dashboard, or **File** → **New** → **Terminal** inside JupyterLab, and write a Slurm script.

1. Save your program as a .py file

For example, save the example above as `bell_state.py` in your scratch folder.

2. Create a job script

Create a file called `run_q50.sh`:

```
#!/bin/bash
#SBATCH --job-name=q50-bell
#SBATCH --account=project_465003017
#SBATCH --reservation=JQH2026
#SBATCH --partition=debug
#SBATCH --time=00:15:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --output=q50-%j.out

module load Local-quantum
module load fiqci-vtt-qiskit-JQH

python bell_state.py
```

3. Submit and monitor

```
sbatch run_q50.sh      # submit the job
squeue --me           # see your queued / running jobs
cat q50-<jobid>.out    # read the output once it finishes
scancel <jobid>        # cancel a job if needed
```

NOTE

The `--account`, `--reservation` and `--partition` lines must match the event values for the reservation to apply. Drop the `--reservation` line if you want to run on a partition other than debug.

Installing extra Python packages (Not recommended)

The event module already includes Qiskit and the Q50 tools. If you need an additional package, install it into **your own user space** with the `--user` flag so you do not touch the shared system environment.

In a notebook cell:

```
pip install --user matplotlib
```

Or from a terminal (after loading the modules):

```
module load Local-quantum
module load fiqci-vtt-qiskit-JQH
pip install --user matplotlib
```

IMPORTANT

Please avoid installing several packages with this method as it puts a strain on the file system. After installing, restart the notebook kernel (**Kernel** → **Restart Kernel**) so the new package is picked up.

TIP

Want to extend the current module? Need several extra packages? Consider using [LUMI container wrapper](#) to keep dependencies isolated and reproducible. After creating your own module, setup you environment by loading first `fiqci-vtt-standard/v1.0-fiqci-junction-default` and then load your own module.

Error Mitigation with FiQCI-EMS

The **FiQCI Error Mitigation Service (EMS)** is a Python library that wraps IQM quantum backends and applies **error mitigation**. EMS lets you run circuits with improved accuracy by specifying a **mitigation level**. See [FiQCI-EMS docs](#) for full details.

The package is already available in the event module (`fiqci-vtt-qiskit-JQH`).

Using EMS is a small change to the example from Step 4 (Run your first quantum program): instead of running on the bare backend, you wrap it with EMS and pass a mitigation level.

```
import os
from qiskit import QuantumCircuit, transpile
from iqm.qiskit_iqm import IQMProvider
from fiqci.ems import FiQCIBackend

Q50_CORTEX_URL = os.getenv('Q50_CORTEX_URL')
provider = IQMProvider(Q50_CORTEX_URL, quantum_computer='q50')
backend = provider.get_backend()

ems_backend = FiQCIBackend(backend, mitigation_level=1) # specify the mitigation level

circuit = QuantumCircuit(2)
circuit.h(0)
circuit.cx(0, 1)
circuit.measure_all()

transpiled = transpile(circuit, ems_backend)
job = ems_backend.run(transpiled, shots=1000)

result = job.result()
```

TIP

EMS provides utilities for calculating the expectation values and supports readout error mitigation (REM), pauli twirling, dynamic decoupling and zero noise extrapolation (ZNE). You can choose to explicitly set the **calibration file** when initialising an EMS backend. This will save the REM data to the file and avoid running calibration circuits each time you submit a circuit.

FAQ & troubleshooting

My session is stuck in “Queued” / takes a long time to start.

Make sure you selected the `JQH2026` reservation and the `debug` partition, these give you priority. Outside the reservation, jobs share LUMI partitions other users and may wait longer. Jobs with fewer cores and less memory also starts faster.

My session ended and I lost my work.

The debug partition allows 30 minutes per session. Save often (**File** → **Save All**), and store notebooks in your **scratch** folder rather than the temporary session space. Long-running work is better submitted with `sbatch` ([see above](#)).

I get `ModuleNotFoundError` / “No module named qiskit”.

The module lines were probably not loaded. Confirm both `module load Local-quantum` and `module -ignore_cache load fiqci-vtt-qiskit-JQH` are in the “Script or path to script” box at launch, then relaunch.

My quantum job fails or hangs when I call `backend.run()`.

Q50 may be offline for calibration. Check its status at fiqci.fi/status

Useful links

- [CSC - Quantum computing documentation](#)
- [FiQCI Error Mitigation Service \(EMS\)](#)
- [FiQCI device status](#)
- [FiQCI blogs](#)
- [LUMI Slurm partitions](#)
- [LUMI storage & directories](#)